

Министерство образования и науки Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
02.03.01 Математика и компьютерные науки

Отчёт по дисциплине
Вычислительная математика
Лабораторная работа №5
«Численное интегрирование»
Вариант 18

Выполнил:
студент группы 5130201/40003
Джабраилов А.В.
Подпись: _____

Принял:
К. физ.-мат. наук, доц., доц.
Пак В.Г.
Подпись: _____
«_____» _____ 2026 г.

Санкт-Петербург, 2026

1 Задание

1. Вычислить приближённо определённый интеграл по простейшим формулам при различных шагах разбиения, оценить погрешности приближения.
2. Вычислить приближённо значение интеграла по формулам Ньютона–Котеса с различными количествами узлов.
3. Вычислить приближённо значение интеграла по формулам Гаусса с различными количествами узлов, оценить погрешности.

2 Цель работы

Освоить квадратурные формулы численного интегрирования, реализовать простейшие формулы, формулы Ньютона–Котеса и формулы Гаусса для заданного варианта, а также выполнить оценку погрешности там, где она задаётся формулами методических указаний.

3 Ход работы

Для варианта 18 необходимо вычислить интеграл от функции

$$y = \operatorname{arcctg} 2^x$$

на отрезке

$$x \in [1; 3].$$

В программе используется равенство

$$\operatorname{arcctg} t = \arctan \frac{1}{t}, \quad t > 0.$$

Так как $2^x > 0$ на всём рассматриваемом отрезке, подынтегральная функция имеет вид

$$f(x) = \arctan \frac{1}{2^x}.$$

Таблица 1: Исходные данные варианта 18.

Параметр	Значение
Функция	$\operatorname{arcctg} 2^x$
Отрезок интегрирования	$[1; 3]$
n для формулы Гаусса	5
n для формулы Ньютона–Котеса	4

Для выполнения лабораторной работы была создана программа на языке *Haskell*. В программе не используются готовые численные интеграторы и не используется адаптивный метод. Все вычисления выполнены по формулам, приведённым в методических указаниях.

3.1 Простейшие квадратурные формулы

Ниже приведён фрагмент кода, реализующий простейшие квадратурные формулы.

Listing 1: Фрагмент кода простейших квадратурных формул

```
1 leftRect n func l r =  
2   let h = (r - l) / fromIntegral n
```

```

3      xs = [l + fromIntegral i * h | i <- [0 .. n - 1]]
4      in h * sum (map func xs)
5
6 rightRect n func l r =
7     let h = (r - l) / fromIntegral n
8     xs = [l + fromIntegral i * h | i <- [1 .. n]]
9     in h * sum (map func xs)
10
11 middleRect n func l r =
12     let h = (r - l) / fromIntegral n
13     xs = [l + (fromIntegral i + 0.5) * h | i <- [0 .. n - 1]]
14     in h * sum (map func xs)
15
16 trapezoid n func l r =
17     let h = (r - l) / fromIntegral n
18     innerSum = sum [func (l + fromIntegral i * h) | i <- [1 .. n - 1]]
19     in h * ((func l + func r) / 2.0 + innerSum)

```

По методическим указаниям отрезок $[a; b]$ разбивается на n частичных отрезков равной длины. Шаг и узлы разбиения имеют вид

$$h = \frac{b - a}{n}, \quad x_i = a + ih, \quad i = 0, 1, \dots, n.$$

Формула левых прямоугольников:

$$I \approx h \sum_{i=0}^{n-1} f(x_i).$$

Подставим $a = 1, b = 3, n = 4$:

$$h = \frac{3 - 1}{4} = 0,5, \quad x_i = 1 + 0,5i.$$

$$I_{\text{left}}^{(4)} = 0,5 (f(1) + f(1,5) + f(2) + f(2,5)) = 0,611716113624.$$

Формула правых прямоугольников:

$$I \approx h \sum_{i=1}^n f(x_i).$$

Подставим $a = 1, b = 3, n = 4$:

$$I_{\text{right}}^{(4)} = 0,5 (f(1,5) + f(2) + f(2,5) + f(3)) = 0,442069806397.$$

Формула центральных прямоугольников:

$$I \approx h \sum_{i=0}^{n-1} f\left(x_i + \frac{h}{2}\right).$$

Подставим $a = 1, b = 3, n = 4$:

$$I_{\text{middle}}^{(4)} = 0,5 (f(1,25) + f(1,75) + f(2,25) + f(2,75)) = 0,520882771903.$$

Формула трапеций:

$$I \approx h \left(\frac{f(a) + f(b)}{2} + \sum_{i=1}^{n-1} f(x_i) \right).$$

Подставим $a = 1, b = 3, n = 4$:

$$I_{\text{trap}}^{(4)} = 0,5 \left(\frac{f(1) + f(3)}{2} + f(1,5) + f(2) + f(2,5) \right) = 0,526892960010.$$

Формула парабол:

$$I \approx \frac{h}{6} \left(f(a) + f(b) + 4 \sum_{i=0}^{n-1} f\left(x_i + \frac{h}{2}\right) + 2 \sum_{i=1}^{n-1} f(x_i) \right).$$

В программе значение по формуле парабол вычисляется на равномерной сетке с чётным числом промежутков. Для $n = 4$ получено

$$I_{\text{Simp}}^{(4)} = 0,522863958380.$$

Оценки погрешностей простейших формул взяты из методических указаний:

$$|R| \leq \frac{b-a}{2} h M_1, \quad M_1 = \max_{x \in [a;b]} |f'(x)|,$$

$$|R| \leq \frac{b-a}{24} h^2 M_2, \quad M_2 = \max_{x \in [a;b]} |f''(x)|,$$

$$|R| \leq \frac{b-a}{12} h^2 M_2, \quad M_2 = \max_{x \in [a;b]} |f''(x)|,$$

$$|R| \leq \frac{b-a}{180} h^4 M_4, \quad M_4 = \max_{x \in [a;b]} |f^{(4)}(x)|.$$

Вычисленные значения максимумов, использованные в программе:

$$M_1 = 0,2772588722239781, \quad M_2 = 0,12011325347892068,$$

$$M_4 = 0,15733720319422967, \quad M_6 = 0,3128065157082597.$$

3.2 Формулы Ньютона–Котеса

Ниже приведён фрагмент кода, реализующий формулы Ньютона–Котеса.

Listing 2: Фрагмент кода формул Ньютона–Котеса

```

1 newtonCotesData n =
2   case n of
3     1 -> Just (1.0 / 2.0, [1.0, 1.0], 1.0 / 12.0)
4     2 -> Just (1.0 / 3.0, [1.0, 4.0, 1.0], 1.0 / 90.0)
5     3 -> Just (3.0 / 8.0, [1.0, 3.0, 3.0, 1.0], 3.0 / 80.0)
6     4 -> Just (2.0 / 45.0, [7.0, 32.0, 12.0, 32.0, 7.0], 8.0 / 945.0)
7     5 -> Just (5.0 / 288.0, [19.0, 75.0, 50.0, 50.0, 75.0, 19.0], 275.0 /
12096.0)
8     _ -> Nothing
9
10 newtonCotes n func l r =
11   case newtonCotesData n of
12     Just (bn, coeffs, _) ->
13       let h = (r - l) / fromIntegral n
14         xs = [l + fromIntegral i * h | i <- [0 .. n]]
15         in bn * h * sum (zipWith (*) coeffs (map func xs))

```

Формула Ньютона–Котеса из методических указаний:

$$\int_a^b f(x) dx \approx B_n h \sum_{i=0}^n a_i^{(n)} f(x_i),$$

где

$$x_i = a + ih, \quad i = 0, 1, \dots, n, \quad h = \frac{b-a}{n}.$$

Для варианта 18 задано $n = 4$. По таблице коэффициентов:

$$B_4 = \frac{2}{45}, \quad a_0^{(4)} = 7, \quad a_1^{(4)} = 32, \quad a_2^{(4)} = 12, \quad a_3^{(4)} = 32, \quad a_4^{(4)} = 7.$$

Шаг равен

$$h = \frac{3-1}{4} = 0,5.$$

Узлы имеют вид

$$x_0 = 1, \quad x_1 = 1,5, \quad x_2 = 2, \quad x_3 = 2,5, \quad x_4 = 3.$$

Подставим числа в формулу:

$$I_{NC}^{(4)} = \frac{2}{45} \cdot 0,5 (7f(1) + 32f(1,5) + 12f(2) + 32f(2,5) + 7f(3)).$$

$$I_{NC}^{(4)} = 0,522878949915.$$

Остаточный член для $n = 4$ по таблице методических указаний:

$$r_4(h) = -\frac{8h^7}{945} f^{VI}(\xi).$$

Для оценки предельной погрешности используем модуль остаточного члена:

$$|r_4(h)| \leq \frac{8h^7}{945} M_6.$$

Подставим числа:

$$|r_4(h)| \leq \frac{8 \cdot 0,5^7}{945} \cdot 0,3128065157082597 = 2,068826162092 \cdot 10^{-5}.$$

3.3 Формулы Гаусса

Ниже приведён фрагмент кода, реализующий формулы Гаусса.

Listing 3: Фрагмент кода формул Гаусса

```
1 gaussData n =
2   case n of
3     2 -> Just [(-0.577350, 1.0), (0.577350, 1.0)]
4     3 -> Just [(-0.774597, 5.0 / 9.0), (0.0, 8.0 / 9.0), (0.774597, 5.0 /
5       9.0)]
6     4 -> Just [(-0.861136, 0.347855), (-0.339981, 0.652145),
7       (0.339981, 0.652145), (0.861136, 0.347855)]
8     5 -> Just [(-0.906180, 0.236927), (-0.538469, 0.478629),
9       (0.0, 0.568889), (0.538469, 0.478629),
10      (0.906180, 0.236927)]
```

```

10     _ -> Nothing
11
12 gauss n func l r =
13     case gaussData n of
14     Just pairs ->
15         let mid = (l + r) / 2.0
16             half = (r - l) / 2.0
17             oneTerm (x, weight) = weight * func (mid + half * x)
18             in half * sum (map oneTerm pairs)

```

В методических указаниях таблица узлов и весов Гаусса приведена для интеграла

$$\int_{-1}^1 f(x) dx.$$

Если интеграл вычисляется на отрезке $[a; b]$, то узлы пересчитываются по формуле

$$t_i = \frac{a+b}{2} + \frac{b-a}{2}x_i.$$

Тогда

$$\int_a^b f(t) dt \approx \frac{b-a}{2} \sum_{i=1}^n A_i f(t_i).$$

Для варианта 18 задано $n = 5$. По таблице Гаусса:

x_i	A_i
-0,906180	0,236927
-0,538469	0,478629
0	0,568889
0,538469	0,478629
0,906180	0,236927

Для отрезка $[1; 3]$ имеем

$$\frac{a+b}{2} = 2, \quad \frac{b-a}{2} = 1.$$

Следовательно,

$$t_i = 2 + x_i.$$

Получим узлы на отрезке $[1; 3]$:

$$t_1 = 1,093820, \quad t_2 = 1,461531, \quad t_3 = 2, \quad t_4 = 2,538469, \quad t_5 = 2,906180.$$

Подставим числа в формулу:

$$I_G^{(5)} = 1 \cdot (0,236927f(1,093820) + 0,478629f(1,461531) + 0,568889f(2) + \\ + 0,478629f(2,538469) + 0,236927f(2,906180)).$$

$$I_G^{(5)} = 0,522887989168.$$

4 Результаты

4.1 Простейшие квадратурные формулы

Таблица 2: Результаты по формуле левых прямоугольников.

n	Значение интеграла	Оценка погрешности
4	0,611716113624	$1,386294361120 \cdot 10^{-1}$
8	0,566299442763	$6,931471805599 \cdot 10^{-2}$
16	0,544343483804	$3,465735902800 \cdot 10^{-2}$
32	0,533553116652	$1,732867951400 \cdot 10^{-2}$

Таблица 3: Результаты по формуле правых прямоугольников.

n	Значение интеграла	Оценка погрешности
4	0,442069806397	$1,386294361120 \cdot 10^{-1}$
8	0,481476289150	$6,931471805599 \cdot 10^{-2}$
16	0,501931906998	$3,465735902800 \cdot 10^{-2}$
32	0,512347328249	$1,732867951400 \cdot 10^{-2}$

Таблица 4: Результаты по формуле центральных прямоугольников.

n	Значение интеграла	Оценка погрешности
4	0,520882771903	$2,502359447478 \cdot 10^{-3}$
8	0,522387524845	$6,255898618694 \cdot 10^{-4}$
16	0,522762749500	$1,563974654673 \cdot 10^{-4}$
32	0,522856494709	$3,909936636684 \cdot 10^{-5}$

Таблица 5: Результаты по формуле трапеций.

n	Значение интеграла	Оценка погрешности
4	0,526892960010	$5,004718894955 \cdot 10^{-3}$
8	0,523887865957	$1,251179723739 \cdot 10^{-3}$
16	0,523137695401	$3,127949309347 \cdot 10^{-4}$
32	0,522950222451	$7,819873273367 \cdot 10^{-5}$

Таблица 6: Результаты по формуле парабол.

n	Значение интеграла	Оценка погрешности
4	0,522863958380	$1,092619466627 \cdot 10^{-4}$
8	0,522886167939	$6,828871666416 \cdot 10^{-6}$
16	0,522887638549	$4,268044791510 \cdot 10^{-7}$
32	0,522887731467	$2,667527994694 \cdot 10^{-8}$

4.2 Формулы Ньютона–Котеса

Таблица 7: Результаты по формулам Ньютона–Котеса.

n	Значение интеграла	Оценка погрешности
1	0,588002603548	$8,007550231928 \cdot 10^{-2}$
2	0,522639085352	$1,748191146603 \cdot 10^{-3}$
3	0,522768406589	$7,769738429345 \cdot 10^{-4}$
4	0,522878949915	$2,068826162092 \cdot 10^{-5}$
5	0,522882856072	$1,165162894490 \cdot 10^{-5}$

Для варианта 18 по формуле Ньютона–Котеса с $n = 4$ получено

$$I_{NC}^{(4)} = 0,522878949915, \quad |r_4(h)| \leq 2,068826162092 \cdot 10^{-5}.$$

4.3 Формулы Гаусса

Таблица 8: Результаты по формулам Гаусса.

Количество узлов	Значение интеграла
2	0,523045828497
3	0,522896341738
4	0,522887598540
5	0,522887989168

Для варианта 18 по формуле Гаусса с пятью узлами получено

$$I_G^{(5)} = 0,522887989168.$$

Так как в методических указаниях для таблицы узлов и весов Гаусса не приведён явный остаточный член, в программе выводятся приближённые значения при разных количествах узлов.

5 Вывод

В ходе лабораторной работы были реализованы формулы численного интегрирования для варианта 18. Для функции $f(x) = \operatorname{arctg} 2^x$ на отрезке $[1; 3]$ были вычислены приближённые значения интеграла по простейшим квадратурным формулам, по формулам Ньютона–Котеса и по формулам Гаусса.

При увеличении числа разбиений в простейших формулах значения интеграла стабилизируются около 0,52288. Формулы центральных прямоугольников, трапеций и парабол дают более точные результаты, чем формулы левых и правых прямоугольников. Для заданного варианта получены основные ответы:

$$I_{NC}^{(4)} = 0,522878949915, \quad I_G^{(5)} = 0,522887989168.$$

6 Приложение

Listing 4: Файл Main.hs

```
1 module Main where
2
3 import Text.Printf
4
5 a :: Double
6 a = 1.0
7
8 b :: Double
9 b = 3.0
10
11 f :: Double -> Double
12 f x = atan (1.0 / (2.0 ** x))
13
14 m1 :: Double
15 m1 = 0.2772588722239781
16
17 m2 :: Double
18 m2 = 0.12011325347892068
19
20 m4 :: Double
21 m4 = 0.15733720319422967
22
23 m6 :: Double
24 m6 = 0.3128065157082597
25
26 leftRect :: Int -> (Double -> Double) -> Double -> Double -> Double
27 leftRect n func l r =
28     let h = (r - l) / fromIntegral n
29         xs = [l + fromIntegral i * h | i <- [0 .. n - 1]]
30     in h * sum (map func xs)
31
32 rightRect :: Int -> (Double -> Double) -> Double -> Double -> Double
33 rightRect n func l r =
34     let h = (r - l) / fromIntegral n
35         xs = [l + fromIntegral i * h | i <- [1 .. n]]
36     in h * sum (map func xs)
37
38 middleRect :: Int -> (Double -> Double) -> Double -> Double -> Double
39 middleRect n func l r =
40     let h = (r - l) / fromIntegral n
41         xs = [l + (fromIntegral i + 0.5) * h | i <- [0 .. n - 1]]
42     in h * sum (map func xs)
43
44 trapezoid :: Int -> (Double -> Double) -> Double -> Double -> Double
```

```

45 trapezoid n func l r =
46     let h = (r - l) / fromIntegral n
47         innerSum = sum [func (l + fromIntegral i * h) | i <- [1 .. n - 1]]
48     in h * ((func l + func r) / 2.0 + innerSum)
49
50 simpsonComposite :: Int -> (Double -> Double) -> Double -> Double -> Double
51 simpsonComposite nRaw func l r =
52     let n = if even nRaw then nRaw else nRaw + 1
53         h = (r - l) / fromIntegral n
54         oddSum = sum [func (l + fromIntegral i * h) | i <- [1,3 .. n - 1]]
55         evenSum = sum [func (l + fromIntegral i * h) | i <- [2,4 .. n - 2]]
56     in h / 3.0 * (func l + func r + 4.0 * oddSum + 2.0 * evenSum)
57
58 leftRightErrorBound :: Int -> Double
59 leftRightErrorBound n =
60     let h = (b - a) / fromIntegral n
61     in (b - a) * h / 2.0 * m1
62
63 middleErrorBound :: Int -> Double
64 middleErrorBound n =
65     let h = (b - a) / fromIntegral n
66     in (b - a) * h * h / 24.0 * m2
67
68 trapezoidErrorBound :: Int -> Double
69 trapezoidErrorBound n =
70     let h = (b - a) / fromIntegral n
71     in (b - a) * h * h / 12.0 * m2
72
73 simpsonErrorBound :: Int -> Double
74 simpsonErrorBound nRaw =
75     let n = if even nRaw then nRaw else nRaw + 1
76         h = (b - a) / fromIntegral n
77     in (b - a) * h ** 4.0 / 180.0 * m4
78
79 newtonCotesData :: Int -> Maybe (Double, [Double], Double)
80 newtonCotesData n =
81     case n of
82         1 -> Just (1.0 / 2.0, [1.0, 1.0], 1.0 / 12.0)
83         2 -> Just (1.0 / 3.0, [1.0, 4.0, 1.0], 1.0 / 90.0)
84         3 -> Just (3.0 / 8.0, [1.0, 3.0, 3.0, 1.0], 3.0 / 80.0)
85         4 -> Just (2.0 / 45.0, [7.0, 32.0, 12.0, 32.0, 7.0], 8.0 / 945.0)
86         5 -> Just (5.0 / 288.0, [19.0, 75.0, 50.0, 50.0, 75.0, 19.0], 275.0 /
12096.0)
87         _ -> Nothing
88
89 newtonCotes :: Int -> (Double -> Double) -> Double -> Double -> Double
90 newtonCotes n func l r =
91     case newtonCotesData n of

```

```

92     Nothing -> error "Unsupported Newton-Cotes n"
93     Just (bn, coeffs, _) ->
94         let h = (r - l) / fromIntegral n
95             xs = [l + fromIntegral i * h | i <- [0 .. n]]
96             values = map func xs
97             in bn * h * sum (zipWith (*) coeffs values)
98
99 newtonCotesErrorBound :: Int -> Double
100 newtonCotesErrorBound n =
101     case newtonCotesData n of
102     Nothing -> error "Unsupported Newton-Cotes n"
103     Just (_, _, c) ->
104         let h = (b - a) / fromIntegral n
105         in case n of
106             1 -> c * h ** 3.0 * m2
107             2 -> c * h ** 5.0 * m4
108             3 -> c * h ** 5.0 * m4
109             4 -> c * h ** 7.0 * m6
110             5 -> c * h ** 7.0 * m6
111             _ -> error "Unsupported Newton-Cotes n"
112
113 gaussData :: Int -> Maybe [(Double, Double)]
114 gaussData n =
115     case n of
116     2 -> Just
117         [ (-0.577350, 1.0)
118         , ( 0.577350, 1.0)
119         ]
120     3 -> Just
121         [ (-0.774597, 5.0 / 9.0)
122         , ( 0.0,      8.0 / 9.0)
123         , ( 0.774597, 5.0 / 9.0)
124         ]
125     4 -> Just
126         [ (-0.861136, 0.347855)
127         , (-0.339981, 0.652145)
128         , ( 0.339981, 0.652145)
129         , ( 0.861136, 0.347855)
130         ]
131     5 -> Just
132         [ (-0.906180, 0.236927)
133         , (-0.538469, 0.478629)
134         , ( 0.0,      0.568889)
135         , ( 0.538469, 0.478629)
136         , ( 0.906180, 0.236927)
137         ]
138     _ -> Nothing
139

```

```

140 gauss :: Int -> (Double -> Double) -> Double -> Double -> Double
141 gauss n func l r =
142     case gaussData n of
143         Nothing -> error "Unsupported Gauss n"
144         Just pairs ->
145             let mid = (l + r) / 2.0
146                 half = (r - l) / 2.0
147                 oneTerm (x, weight) = weight * func (mid + half * x)
148                 in half * sum (map oneTerm pairs)
149
150 printLine :: IO ()
151 printLine = putStrLn "
152     -----"
153
154 printSimpleResult :: String -> Int -> Double -> Double -> IO ()
155 printSimpleResult name n value bound = do
156     printf "%-20s n = %-4d value = %.12f error_bound = %.12e\n"
157         name n value bound
158
159 printNCResult :: Int -> Double -> Double -> IO ()
160 printNCResult n value bound = do
161     printf "Newton-Cotes          n = %-4d value = %.12f error_bound = %.12e\n"
162         n value bound
163
164 printGaussResult :: Int -> Double -> IO ()
165 printGaussResult n value = do
166     printf "Gauss                  n = %-4d value = %.12f\n"
167         n value
168
169 main :: IO ()
170 main = do
171     putStrLn "Lab 5. Numerical integration"
172     putStrLn "Variant 18"
173     putStrLn "Function: y = arccot(2^x)"
174     putStrLn "Interval: [1; 3]"
175     printLine
176
177     putStrLn "Task 1. Simple composite formulas with different partition steps"
178     putStrLn "Columns: formula, n, approximate value, theoretical error bound"
179     printLine
180
181     let parts = [4, 8, 16, 32]
182
183     mapM_ (\n -> do
184         let value = leftRect n f a b
185         let bound = leftRightErrorBound n

```



```

185         printSimpleResult "Left rectangles" n value bound) parts
186
187     printLine
188
189     mapM_ (\n -> do
190         let value = rightRect n f a b
191         let bound = leftRightErrorBound n
192         printSimpleResult "Right rectangles" n value bound) parts
193
194     printLine
195
196     mapM_ (\n -> do
197         let value = middleRect n f a b
198         let bound = middleErrorBound n
199         printSimpleResult "Middle rectangles" n value bound) parts
200
201     printLine
202
203     mapM_ (\n -> do
204         let value = trapezoid n f a b
205         let bound = trapezoidErrorBound n
206         printSimpleResult "Trapezoid" n value bound) parts
207
208     printLine
209
210     mapM_ (\n -> do
211         let value = simpsonComposite n f a b
212         let bound = simpsonErrorBound n
213         printSimpleResult "Simpson" n value bound) parts
214
215     printLine
216
217     putStrLn "Task 2. Newton-Cotes formulas with different n"
218     putStrLn "Columns: n, approximate value, theoretical error bound"
219     printLine
220
221     mapM_ (\n -> do
222         let value = newtonCotes n f a b
223         let bound = newtonCotesErrorBound n
224         printNCResult n value bound) [1 .. 5]
225
226     printLine
227
228     putStrLn "Task 3. Gauss formulas with different number of nodes"
229     putStrLn "Columns: nodes count, approximate value"
230     printLine
231
232     mapM_ (\n -> do

```

```

233         let value = gauss n f a b
234         printGaussResult n value) [2 .. 5]
235
236     printLine
237
238     putStrLn "Variant-specific results"
239     let variantNC = newtonCotes 4 f a b
240     let variantNCBound = newtonCotesErrorBound 4
241     let variantGauss = gauss 5 f a b
242
243     printf "Newton-Cotes n = 4 value = %.12f\n" variantNC
244     printf "Newton-Cotes n = 4 error_bound = %.12e\n" variantNCBound
245     printf "Gauss n = 5 value = %.12f\n" variantGauss
246
247     printLine
248     putStrLn "Done."

```